Université CY Cergy Paris Université
Master 1

TD 1 : Wargame

Rapport de travaux dirigés

Auteur : CHEMIM Massiva
Tuteur : Roberto SAUTTO
Date : 25 octobre 2026

Table des matières

1 Le projet et le jeu **2**

2 Les fonctions demandées **2**

2.1 `f_eval` : donner une note à un plateau . . . 2

2.2 `f_max` et `f_min` : Minimax avec coupures . . . 3

2.3 `f_IA` : choisir et jouer le coup . . . 3

2.4 Fonction ajoutée : `f_bataille_plateau` . . . 3

3 Analyse des coupures alpha/bêta **4**

3.1 Principe . . . 4

3.2 Méthode de mesure . . . 4

3.3 Résultats . . . 4

4 Conclusion et limites **4**

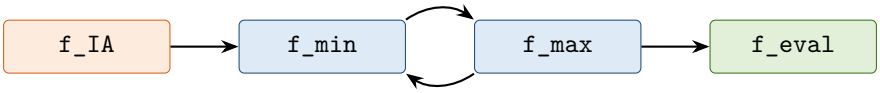
À retenir

Tous les chiffres de ce rapport viennent de **mon programme** `wargame.c`, avec le compteur de nœuds et le chronomètre que j'y ai ajoutés dans `f_IA`. Ils proviennent de deux parties complètes IA contre IA (mode 3), l'une avec les coupures et l'autre sans.

1 Le projet et le jeu

Il consiste à programmer une IA pour un wargame sur un plateau de 10×10. À chaque tour, un joueur déplace **une** unité d'une case dans l'une des 8 directions. Si la case visée contient une unité ennemie, il y a un combat : on additionne les valeurs des unités de chaque camp sur les 8 cases voisines de la case visée, et l'attaquant ne l'emporte que si sa somme est *strictement supérieure* (en cas d'égalité, le défenseur gagne). On gagne en atteignant la ligne de fond adverse (ligne 0 pour les `x`, ligne 9 pour les `o`) ou en éliminant toutes les unités de l'adversaire.

L'IA utilise Minimax avec coupures alpha/bêta à la profondeur 3, imposée pour le tournoi. Les fonctions `f_eval`, `f_max`, `f_min` et `f_IA` du patron sont complétées, et une fonction a été ajoutée (`f_bataille_plateau`).



alternent jusqu'à la profondeur 0

2 Les fonctions demandées

2.1 `f_eval` : donner une note à un plateau

`f_eval(jeu, joueur)` renvoie un entier qui dit si la position est bonne pour `joueur`, le joueur de l'IA. Elle teste d'abord la fin de partie : `INFINI` (10000) si un de mes pions a atteint la ligne de fond adverse ou si l'adversaire n'a plus de pions, `-INFINI` dans le cas inverse. Sinon, elle calcule trois différences « moi moins l'adversaire » :

- `force` : somme des valeurs de mes pions moins celle des pions adverses (`f_valeur`) ;
- `nombre` : mon nombre de pions moins celui de l'adversaire (`f_nbPions`) ;
- `avancee` : somme des lignes déjà parcourues par mes pions vers la ligne adverse, moins la même somme pour l'adversaire.

La note finale est la somme pondérée de mon programme :

```
return 10 * force + 3 * nombre + 5 * avancee;
```

Exemple : au départ, les deux camps ont les mêmes forces, nombres et avancées, donc la note est 0. Si un de mes pions gagne un combat contre un pion adverse de valeur v, `force` augmente de v et `nombre` de 1, donc la note augmente de $10v + 3$ (13, 23 ou 33 pour $v = 1, 2, 3$). Si un de mes pions avance d'une ligne, la note augmente de 5.

Pourquoi ce choix ?

Force : ce sont les valeurs qui décident des combats, donc c'est le critère principal (poids 10). **Nombre** : à force égale, plus de pions donnent plus de coups possibles et plus de soutien dans les combats (poids 3). **Avancée** : sans ce critère, l'IA n'a aucune raison de progresser vers la ligne adverse tant que les forces sont égales, alors que c'est la condition de victoire (poids 5).
Ordre de grandeur des poids. Une avancée d'une ligne (5) vaut moins que la capture du plus petit pion adverse (13), donc l'IA ne sacrifie pas un pion pour gagner une ligne. En revanche, cinq lignes de progression d'un seul pion ($5 \times 5 = 25$) valent plus que la capture d'un pion de valeur 2 (23) : un pion qui avance sans opposition devient une menace sérieuse. Je n'ai pas fait de tournoi systématique pour optimiser ces poids : ils sont choisis par cette échelle de grandeurs (voir les limites en section 4).

2.2 `f_max` et `f_min` : Minimax avec coupures

`f_max` traite les coups de l'IA et garde la **meilleure** note ; `f_min` traite ceux de l'adversaire et garde la **pire** pour l'IA, puisqu'il choisira ce qui l'arrange le plus. Leur signature est

```
int f_min(Pion *plateau, int joueur_ia, int profondeur, int alpha, int beta)
```

(et de même pour `f_max`). Pour chaque pion du joueur concerné et chacune des 8 directions, la fonction copie le plateau (`f_copie_plateau`), joue le coup sur la copie avec `f_bouge_piece` (les coups illégaux sont ignorés), puis appelle l'autre fonction avec `profondeur - 1`. Elle s'arrête et renvoie `f_eval` quand la profondeur est nulle, quand la partie est finie (note $\pm$`INFINI`) ou quand aucun coup n'est possible. La note est toujours calculée du point de vue de `joueur_ia`.

Les coupures se trouvent dans la boucle : `f_max` met à jour `alpha` et `f_min` met à jour `beta`, et chacune sort de la boucle dès que `alpha >= beta` :

```
if (score < meilleur) meilleur = score;      /* f_min */
if (meilleur < beta)  beta = meilleur;
if (alpha >= beta)    return meilleur;       /* coupure */
```

Pourquoi ce choix ?

Copie du plateau : chaque coup est joué sur une copie locale (`Pion copie[NB_LIGNES * NB_COLONNES]` sur la pile, sans `malloc`), donc il n'y a rien à annuler au retour de la récursion et le plateau du parent reste intact. **Note toujours du point de vue de `joueur_ia`** : toutes les notes de l'arbre sont comparables, et `f_min` n'a qu'à prendre le minimum. **Compteur `nb_coups`** : si aucun coup n'est possible dans un nœud, on renvoie sa note plutôt que `INFINI` ou `-INFINI`, qui n'auraient pas de sens.

2.3 `f_IA` : choisir et jouer le coup

`f_IA(joueur)` est la racine de l'arbre : elle essaie tous les coups légaux du joueur, simule chacun sur une copie et appelle

```
score = f_min(copie, joueur, 2, -INFINI, INFINI);
```

Les trois niveaux sont donc : mon coup (racine), la réponse de l'adversaire (`f_min`, profondeur 2), puis mon coup suivant (`f_max`, profondeur 1), c'est-à-dire la profondeur 3 . Le meilleur coup est mémorisé puis joué sur le vrai plateau `plateauDeJeu`.

Pourquoi ce choix ?

La variable `coup_trouve` : `meilleur_score` vaut `-INFINI` au départ. Si tous les coups mènent à une défaite (note `-INFINI`), le test `score > meilleur_score` ne serait jamais vrai et l'IA ne jouerait aucun coup (elle passerait son tour). Avec `|| !coup_trouve`, le premier coup est toujours retenu, puis seuls de meilleurs coups le remplacent.

2.4 Fonction ajoutée : `f_bataille_plateau`

La fonction `f_bataille` du patron lit le tableau global `plateauDeJeu`, même quand `f_bouge_piece` travaille sur une copie. Dans Minimax, tous les coups sont joués sur des copies : les combats auraient été calculés avec les pions voisins du *vrai* plateau et les notes de l'arbre auraient été fausses. J'ai donc ajouté `f_bataille_plateau(plateau, l, c)`, qui applique la même règle (somme des voisins, égalité en faveur du défenseur) sur le plateau reçu en paramètre, et `f_bouge_piece` l'appelle à la place de `f_bataille`.

3 Analyse des coupures alpha/bêta

3.1 Principe

`alpha` est la meilleure note que l'IA est déjà sûre d'obtenir, `beta` la pire que l'adversaire peut lui imposer. Dès que $\alpha \geq \beta$, la suite du nœud ne peut plus changer le choix et on arrête la boucle. Le résultat est identique à Minimax seul.

3.2 Méthode de mesure

J'ai ajouté à `wargame.c` une variable globale `long nb_noeuds`, incrémentée au début de `f_min` et de `f_max` (donc à chaque nœud visité), remise à zéro au début de `f_IA`, et un chronomètre `clock()` autour de la recherche. `f_IA` affiche à chaque coup une ligne du type `IA : 5431 noeuds visites, 0.0110 s`. J'ai lancé une partie complète IA contre IA (mode 3, profondeur 3), une première fois normalement, puis une seconde fois en remplaçant les deux tests `if (alpha >= beta)` par `if (0)` (coupures désactivées). Les deux parties comptent **104 coups**, avec la même suite de plateaux et le même vainqueur (o) : les coupures ne changent pas le coup choisi.

3.3 Résultats

Mesure	Sans coupures	Avec coupures	Gain
Premier coup : nœuds	81 732	5 431	×15,0
Premier coup : temps	0,177 s	0,011 s	×16
Partie : nœuds au total	16 062 018	3 049 332	×5,3
Partie : temps de calcul total	31,70 s	6,07 s	×5,2
Nœuds par coup (moyenne)	154 442	29 320	×5,3

Observations.

— **Les coupures divisent le travail par 5 sur toute la partie**, et ce gain reste stable (entre ×4,9 et ×5,7 par tranche de 26 coups). Le tout premier coup est plus favorable (×15).

— **Le temps suit le nombre de nœuds** : quand les nœuds sont divisés par 5, le temps l'est aussi. Les temps dépendent de mon ordinateur, pas le nombre de nœuds.

À retenir

Les coupures alpha/bêta réduisent le nombre de nœuds d'un facteur 5 sur une partie complète (15 sur le premier coup) sans changer la partie jouée : les 104 coups sont identiques avec et sans coupures. Une partie entière à la profondeur 3 passe de 31,7 s à 6,1 s de calcul.

4 Conclusion et limites

L'IA joue des parties complètes et légales avec Minimax à la profondeur 3, et les coupures alpha/bêta divisent le nombre de nœuds par 5 sur une partie sans changer le résultat.

Limites :

— Les poids (10, 3, 5) sont choisis par raisonnement.

— Les mesures portent sur une seule partie à la profondeur 3, sur ma machine.

Piste d'amélioration : Dans `f_IA`, l'appel à `f_min` utilise toujours `alpha = -INFINI` ; utiliser `meilleur_score` permettrait plus de coupures dès la racine.